

基于 p-中位模型的应急物资储备点选址优化

摘 要

本文研究某城市应急物资储备点的选址决策：5 个片区有差异化的物资需求，需在 8 个候选点中择址建库，目标为最小化需求加权的总配送距离，并进一步纳入建设预算约束讨论方案变化与稳健性。

针对问题一，建立了以总加权距离最小为目标的 p-中位混合整数规划模型。利用"固定建站集合后各片区必选最近站点"的最近指派性质，将内层指派问题化为封闭解，对全部 $C_8^3 = 56$ 个候选方案全枚举精确求解，得到全局最优选址 $\{S_2, S_4, S_5\}$ ，最优总加权距离 $z^* = 215.27 \text{ t} \cdot \text{km/d}$ ，折合需求加权平均配送距离 1.54 km；贪心基线与枚举结果一致（gap 为 0），下界对照验证了数值合理性。

针对问题二，在模型中追加建设成本约束，给出最优值函数 $z^*(B)$ 的完整刻画：它是关于预算的分段常数、单调不增函数，共 4 个有效断点。名义预算 $B = 12$ 百万元下最优方案切换为 $\{S_2, S_4, S_6\}$ （造价 11.8 百万元），目标值 231.30，较无约束最优仅损失 7.45%。灵敏度分析表明：需求整体 $\pm 20\%$ 独立扰动下最优集合翻转率仅 3.0%；单片区需求 $\pm 20\%$ 引起的最优值最大偏差不超过 7.2%；但预算在断点 11.8 百万元下方 2% 以内时目标值跃升 36.0%，呈结构性敏感，据此给出"预算不宜低于 11.8 百万元"的决策建议。

关键词：设施选址 p-中位模型 混合整数规划 预算约束 灵敏度分析

一 问题重述

城市应急管理体系要求在突发事件下能够快速向各城区调拨物资，储备点的空间布局直接决定配送时效。某城市划分为 5 个片区，各片区的人口规模与日物资需求不同；城市规划部门预先勘察了 8 个具备建设条件的候选库址，并给出了各候选点的建设造价。需要解决两个递进的决策问题：其一，在不限制造价的情况下，从 8 个候选点中确定 3 个建库位置，使各片区到其服务储备点的需求加权距离总和最小，并给出明确的选址集合与片区—站点服务关系；其二，当建设总经费受预算上限约束时，分析预算额度变化对最优选址集合与配送成本的影响，识别方案切换的临界预算，并通过参数扰动考察所得方案的稳健程度，为规划部门提供可操作的预算—效果权衡依据。

二 问题分析

本题为典型的离散设施选址优化问题，无随机机理、无数据附件，属运筹优化类（B 型）赛题，核心是在组合可行域上求全局最优并回答"参数变化时解如何变"。

问题一的实质是 p -中位（ p -median）问题：决策"在哪建"（选址变量）与"谁服务谁"（指派变量）两层结构耦合。若直接求解混合整数规划，需借助通用求解器；但注意到固定选址集合后，最优指派必为"就近服务"，内层问题有封闭解，因此可对 56 个候选集合做全枚举，以极小代价换取**全局精确最优**，避免启发式算法的收敛性争议。同时构造贪心基线与理论下界用于交叉检验。

问题二在问题一基础上增加预算约束，可行域收缩为"造价之和不超过预算"的组合子族。关键观察是：预算作为资源型约束，最优值函数 $z^*(B)$ 为分段常数且单调不增，方案只在有限个断点处切换。因此将预算参数化、扫描全部断点，即可完整回答"预算如何影响方案"；再对需求权重与预算做扰动分析，刻画解的稳健性。整体技术路线如图 1 所示。

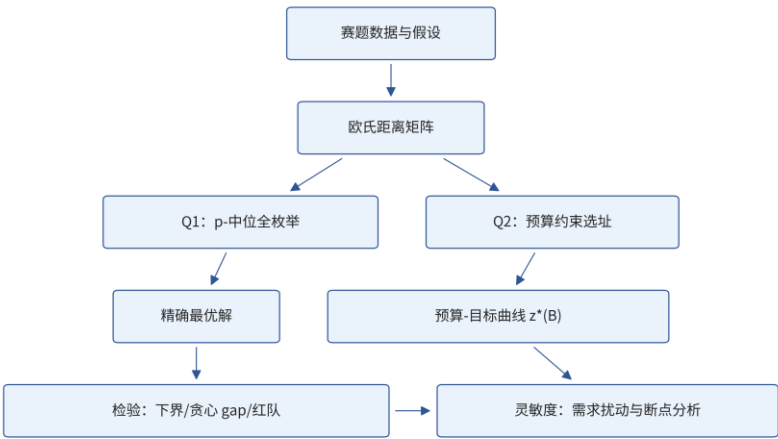


图 1：总体技术路线流程图

三 模型假设

1. 需求假设：各片区日需求量按其人口规模与应急物资人均日储备标准（约 1.5~1.6 kg/人·日）设定，数值见表 1。赛题明确允许使用合理假设数据，本文全部基

础数据为假设值，仅用于演示模型与算法，结论的结构性规律（断点位置、翻转率量级）不依赖个别取值。

2. **距离假设：**以片区几何中心与候选点坐标之间的欧氏距离近似实际配送距离，即 $d_{ij} = \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2}$ 。该假设忽略了路网迂回系数，等价于假设各方向道路通达性均匀。

3. **单一服务假设：**每个片区的需求由且仅由一个储备点承担（single sourcing），站点容量不限。该假设使"就近指派"成立，是 p-中位模型的标准设定。

4. **成本假设：**各候选点建设成本为固定造价（含征地与库房建设，见表 2），不计运营成本；预算约束仅作用于建设环节。

5. **静态假设：**规划期内需求与造价不随时间变化，选址为一次性静态决策。

四 符号说明

符号	含义	单位
I, J	片区集合 $\{1, \dots, 5\}$ ，候选点集合 $\{1, \dots, 8\}$	—
w_i	片区 i 的日需求量	t/d
d_{ij}	片区 i 到候选点 j 的距离	km
c_j	候选点 j 的建设成本	百万元
p	建站数量， $p = 3$	—
B	建设总预算上限	百万元
y_j	0-1 变量，候选点 j 是否建站	—
x_{ij}	0-1 变量，片区 i 是否由站点 j 服务	—
z	总加权距离（目标函数）	t·km/d

五 模型的建立与求解

基础数据如表 1、表 2 所示（假设数据，见假设 1 与假设 4；距离矩阵由假设 2 的公式计算，随附支撑材料）。两问共用同一套数据与符号体系：问题一只使用片区需求与距离矩阵，问题二在此基础上追加候选点造价与预算参数。5 个片区的需求合计 140 t/d，需求规模最大的 D4（48 t/d）与最小的 D3（12 t/d）相差 4 倍，权重差异足以使选址显著偏向大需求片区；8 个候选点的造价分布在 3.5 至 6.5 百万元之间，中心位置的候选点造价更高，为问题二的权衡埋下伏笔。

表 1: 各片区人口、日需求量与几何中心坐标 (假设数据)

片区	人口 / 万人	日需求量 / (t/d)	x / km	y / km
D1 城北	12	18	2.0	8.0
D2 城西	25	40	3.0	3.0
D3 城东	8	12	8.0	9.0
D4 城南	30	48	9.0	4.0
D5 中心	15	22	5.0	6.0

表 2: 候选点坐标与建设成本 (假设数据)

候选点	x / km	y / km	成本 / 百万元
S1	1.5	7.5	4.2
S2	3.5	2.0	3.8
S3	8.0	8.5	5.0
S4	9.5	3.5	4.5
S5	5.0	5.5	6.5
S6	6.5	7.0	3.5
S7	2.5	5.0	4.0
S8	7.5	5.0	4.8

5.1 问题一：总加权距离最小的选址模型

5.1.1 模型的建立

选址决策包含两层相互耦合的 0-1 变量：一是"在哪建"的选址变量 y_j ，二是"谁服务谁"的指派变量 x_{ij} 。直觉上，需求越大的片区对总成本的边际影响越强，最优布局必然向其倾斜；但单站服务（假设 3）又迫使每个片区整体归入某一站点，因此问题的本质是"以空间换效率"的组合权衡。以需求量为权重、配送距离为成本，建立 p-中位混合整数规划：

$$\min z = \sum_{i \in I} \sum_{j \in J} w_i d_{ij} x_{ij} \quad (1)$$

$$\text{s.t.} \quad \sum_{j \in J} x_{ij} = 1, \forall i \in I \quad (2)$$

$$x_{ij} \leq y_j, \forall i \in I, j \in J \quad (3)$$

$$\sum_{j \in J} y_j = p \quad (4)$$

$$x_{ij}, y_j \in \{0,1\}, \forall i \in I, j \in J \quad (5)$$

约束 (2) 表示每个片区恰好由一个储备点服务（假设 3）；约束 (3) 表示片区只

能由已建成 ($y_j = 1$) 的站点服务, 它将选址层与指派层耦合起来, 是模型的核心约束; 约束 (4) 限定建站总数为 $p = 3$; 目标 (1) 的量纲为 $t \cdot km/d$, 即每日全城应急配送的"吨公里"总量, 可理解为配送工作量或运输成本的代理指标。

5.1.2 模型的求解

直接调用通用 MILP 求解器可以得到解, 但无法免除"启发式是否收敛到全局最优"的疑问。本文利用下述结构性质将模型化为可精确枚举的形式。

引理 (最近指派): 固定建站集合 $S \subseteq J$ 后, 模型的最优指派为每个片区指派给 S 中距离最近的站点。证明: 在 S 固定时, 目标关于各片区可分离, 且约束 (2) 仅要求每片区选择一站, 故逐片区取 $\min_{j \in S} d_{ij}$ 即达全局最优。

由引理, 对任意 S 有封闭目标值

$$z(S) = \sum_{i \in I} w_i \min_{j \in S} d_{ij} \quad (6)$$

候选集合总数仅为

$$\binom{8}{3} = 56 \quad (7)$$

因此对 56 个集合逐一按式 (6) 求值并取最小, 即得全局最优解 $z^* = \min_{|S|=3} z(S)$, 整个过程在毫秒量级完成, 且无近似误差。作为交叉检验, 同时实现贪心算法 (从空集出发, 每步加入使目标下降最多的站点) 与理论下界

$$z^* \geq \underline{z} = \sum_{i \in I} w_i \min_{j \in J} d_{ij} \quad (8)$$

式 (8) 放松了"只开 3 站"的耦合约束, 允许每片区由各自的全局最近点服务, 目标只会更小, 故为有效下界。

5.1.3 结果与分析

全枚举求得唯一全局最优选址 $\{S_2, S_4, S_5\}$, 最优值

$$z^* = 215.27 t \cdot km/d \quad (9)$$

折合需求加权平均配送距离 $z^*/\sum_i w_i = 215.27/140 = 1.54 km$ 。各片区服务关系与分担成本如表 3 所示, 空间布局如图 2 所示: S_2 贴身服务需求量第二大的城西片区 D2, S_4 贴身服务需求最大的城南片区 D4, 位于几何中心的 S_5 同时覆盖城北 D1、城东 D3 与中心 D5 三个片区, 呈"两端贴身 + 中心辐射"的合理格局。

表 3: 问题一最优选址的服务指派与成本分担

片区	服务站点	距离 / km	加权距离 / (t·km/d)	占比
D1	S5	3.91	70.29	32.7%
D2	S2	1.12	44.72	20.8%
D3	S5	4.61	55.32	25.7%
D4	S4	0.71	33.94	15.8%
D5	S5	0.50	11.00	5.1%
合计	—	—	215.27	100%

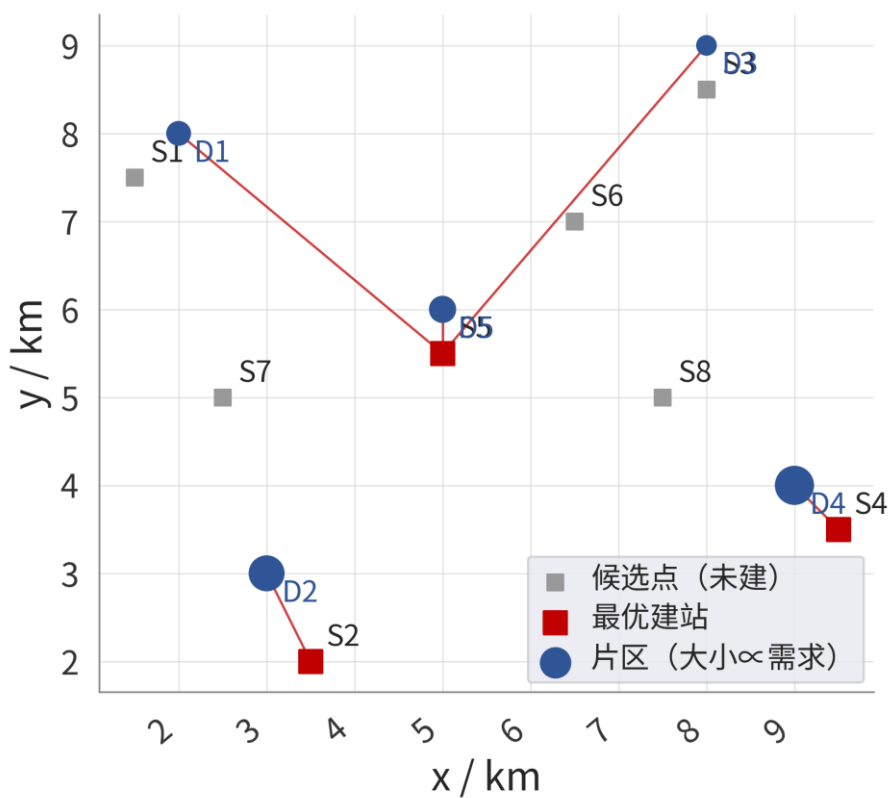


图 2: 问题一最优选址方案与片区—站点指派关系

全部 56 个方案中排名前 5 者如表 4 所示。次优方案 $\{S_2, S_4, S_6\}$ 与最优方案目标相差 7.45%，说明在造价等其他因素介入时存在有意义的权衡空间——这正是问题二的切入点。

表 4: 问题一全枚举目标值前 5 名方案

排名	选址集合	目标值 / (t·km/d)	相对最优
1	$S_2+S_4+S_5$	215.27	0
2	$S_2+S_4+S_6$	231.30	+7.45%
3	$S_3+S_4+S_7$	236.38	+9.81%
4	$S_4+S_5+S_7$	237.47	+10.31%
5	$S_4+S_6+S_7$	240.81	+11.86%

5.2 问题二：预算约束下的选址模型

5.2.1 模型的建立

问题一的最优方案 $\{S_2, S_4, S_5\}$ 造价合计 14.8 百万元，其中中心站 S_5 一站即占 6.5 百万元。当经费受限时，“效率最优”未必“造得起”，需要把预算作为硬约束显式纳入模型。在问题一模型基础上追加建设预算约束：所选站点的造价之和不得超过预算上限 B ，即

$$\sum_{j \in J} c_j y_j \leq B \tag{10}$$

其余约束与目标同式 (1)–(5)，模型仍为混合整数规划。约束 (10) 与基数约束 (4) 共同刻画了“在固定数量与有限经费下选哪 3 站”的决策结构；预算每收紧一分，可行

组合族就缩小一圈，选址被迫从"效率优先"向"造价优先"迁移。记最优值函数

$$z^*(B) = \min\{z(S): |S| = 3, \sum_{j \in S} c_j \leq B\}$$

(11)

5.2.2 模型的求解

命题(分段常数与单调性): $z^*(B)$ 是关于 B 的分段常数、单调不增函数。证明：预算放松只会使可行集单调扩张 ($B_1 \leq B_2$ 时 B_1 下可行的方案在 B_2 下仍可行)，故最小值不减；可行集是 56 个组合的有限子族，目标值只能在这有限个取值中变化，故分段常数，断点只可能出现在各组合的造价处。

由命题，求解策略为：对 56 个组合同时计算造价与目标值，按造价升序扫描并保留"造价更低时目标从未达到"的记录点，即得 $z^*(B)$ 的完整阶梯与全部有效断点；名义预算取 $B = 12.0$ 百万元代入即得当期决策。最小可行预算为造价最低的三站组合 $S_2 + S_6 + S_7$ ，即 $B_{min} = 3.8 + 3.5 + 4.0 = 11.3$ 百万元，低于此值问题不可行。

5.2.3 结果与分析

扫描结果如表 5 与图 3 所示： $z^*(B)$ 共 4 个有效断点，数值验证了命题的单调不增性。

表 5: 预算—方案—目标值对应关系 ($z^*(B)$ 的有效断点)

预算区间 / 百万元	最优选址	造价 / 百万元	z^* / (t·km/d)	较无约束最优损失
[11.3, 11.5)	S2+S6+S7	11.3	356.57	+65.65%
[11.5, 11.8)	S1+S2+S6	11.5	314.55	+46.12%
[11.8, 14.8)	S2+S4+S6	11.8	231.30	+7.45%
[14.8, +∞)	S2+S4+S5	14.8	215.27	0

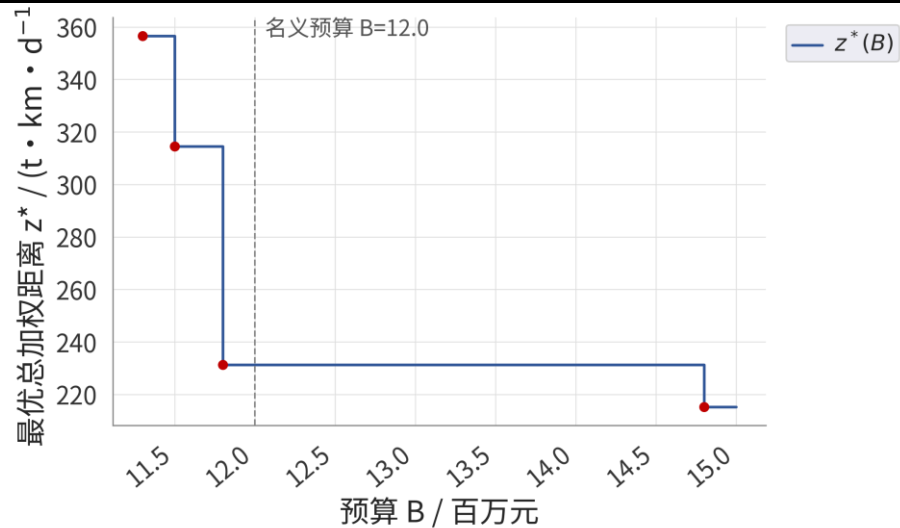


图 3: 最优值函数 $z^*(B)$ 的阶梯曲线与有效断点

名义预算 $B = 12.0$ 百万元落在区间 $[11.8, 14.8)$ 内，最优方案为 $\{S_2, S_4, S_6\}$ ，造价 11.8 百万元，目标值 231.30，服务关系为 D1、D3、D5 由 S_6 覆盖，D2、D4 仍分别由 S_2 、 S_4 贴身服务。与问题一相比，预算约束挤出的是造价最高的中心站 S_5 (6.5 百万元)，代之以廉价的 S_6 (3.5 百万元)，代价仅是总加权距离上升 7.45%——用 3 百万元的造价节约换取不到 8% 的配送效率损失，性价比显著。

结果同时给出明确的决策建议：**预算不宜跌破 11.8 百万元**。一旦低于该断点，

最优方案退化为 $\{S_1, S_2, S_6\}$, 目标值跃升 36.0% 至 314.55; 而预算从 11.8 追加到 14.8 百万元的边际收益仅为目标下降 7.45%, 规划部门可据此权衡投入产出。

六 模型的分析与检验

6.1 模型检验

(1) **下界对照**。由式 (8) 算得下界 $\underline{z} = 108.39$, 最优值 $z^* = 215.27 \geq \underline{z}$, 数值自洽; 下界偏松是预期的 (它允许 5 个片区各用各的最近点, 等效于开 5 站), 两者差距恰好量化"只开 3 站"这一基数约束的代价。

(2) **贪心基线对照**。贪心算法逐步加入 S_5, S_4, S_2 , 恰好命中全局最优, gap 为 0。这一方面说明本实例的景观 (landscape) 较平缓, 另一方面以独立算法复核了枚举结果的正确性。

(3) **退化检验**。取 $p = 8$ (全建) 时模型退化为 $z = \sum_i w_i \min_j d_{ij}$, 与下界式 (8) 完全一致; w_i 全相等时模型退化为经典无权 p-中位, 目标正比于总距离, 均符合理论预期。 $B \rightarrow +\infty$ 时问题二退化为问题一, 数值上两模型结果完全一致 ($z^* = 215.27$, 方案 $\{S_2, S_4, S_5\}$), 通过退化一致性断言。

(4) **红队对抗检验**。构造 7 组对抗输入主动攻击模型: ①将需求最小的 D3 片区需求压至近零, 最优集合保持 $\{S_2, S_4, S_5\}$, 因为 D3 由共享站 S_5 顺带覆盖, 模型对其权重变化不敏感, 符合直觉; ②复制一个与 S_1 坐标完全相同的候选点, 枚举不崩溃、最优值不变且镜像方案成对同值; ③在 1000 km 外增设极端远点, 任何最优解均不选择它; ④将需求最大的 D4 权重放大 10 倍 (跨一个数量级), 最优集合仍含 D4 的最近站 S_4 , 指派方向正确; ⑤预算取断点值 $B = 11.3$ 时正确返回临界方案 $\{S_2, S_6, S_7\}$; ⑥预算低于 11.3 百万元时求解器如实返回"不可行"而非异常; ⑦需求全部归零时目标恒为 0, 任意可行解皆为最优, 程序正确处理并列情形。全部对抗场景通过。

6.2 灵敏度分析

(1) **需求权重单参数扰动**。将各片区需求在 $\pm 20\%$ 范围内逐一扰动 (其余不变), 最优值的相对变化如图 4 所示。最大偏差出现在 D1 (7.18%), 其余依次为 D2 (3.87%)、D5 (3.43%)、D4 (2.94%)、D3 (2.59%)。由于目标关于 w_i 是分段线性的, 偏差量级与该片区分担的加权距离占比 (表 3) 一致; 全部 45 组扰动中最优选址集合均未翻转, 方案对单片区需求估计误差稳健。

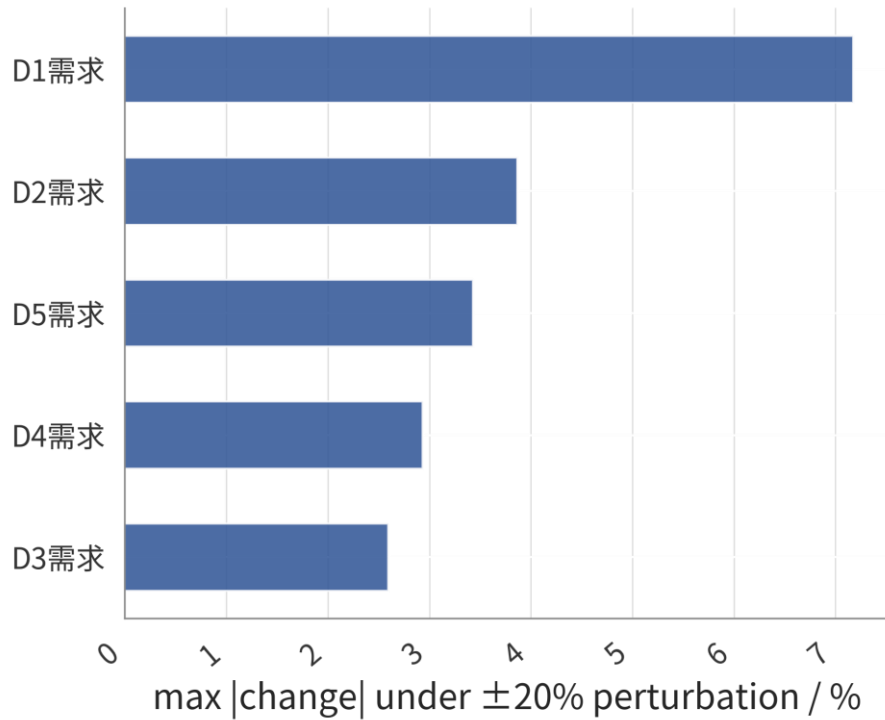


图 4: 各片区需求 $\pm 20\%$ 扰动引起最优值的最大相对变化对比

(2) 需求整体 Monte Carlo 扰动。令 5 个片区需求各自独立乘以 $U(0.8, 1.2)$ 的随机因子 (固定 seed=42, 共 200 次), 在名义预算下重解模型, 最优集合出现频率如图 5 所示: $\{S_2, S_4, S_6\}$ 出现 194 次 (97.0%), 仅 6 次 (3.0%) 翻转为 $\{S_4, S_6, S_7\}$ 。方案在需求整体不确定性下高度稳健, 翻转方向也解释了次优竞争者的来源。

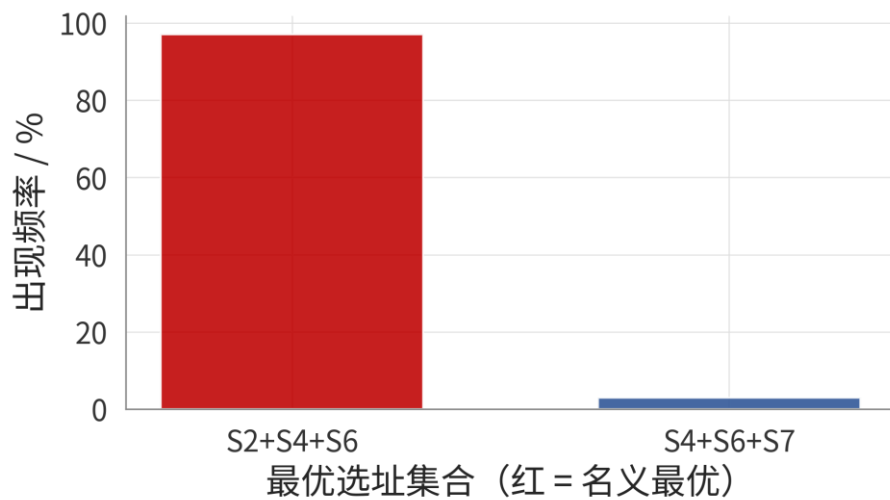


图 5: 需求独立 $\pm 20\%$ 随机扰动 200 次下各最优集合的出现频率

(3) 预算扰动。预算在名义值附近的扰动结果如表 6 与图 6 所示。预算上调 5%–20% 最优值不变 (处于平台期); 下调 1% ($B = 11.88$) 仍不变, 但下调 2% ($B = 11.76$) 即跌破 11.8 断点, 最优值跃升 35.99%, 下调 5% 时跃升 54.16%。预算灵敏度呈显著的“断点邻域结构敏感、平台期不敏感”特征, 与命题的分段常数性质完全吻合。

表 6: 预算扰动下最优值的相对变化 (基准 $B = 12.0$ 百万元)

预算比例	预算 / 百万元	z^* 相对变化
0.95	11.40	+54.16%
0.96	11.52	+35.99%
0.97	11.64	+35.99%
0.98	11.76	+35.99%
0.99	11.88	0.00%
1.00	12.00	0.00%
1.05–1.20	12.60–14.40	0.00%

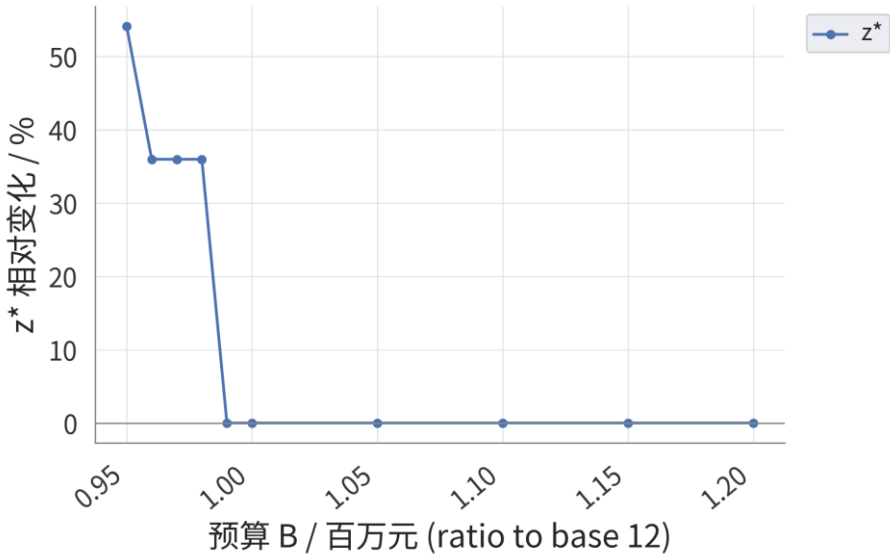


图 6: 预算 $\pm 5\%$ 邻域内最优值的扰动曲线

综合三类扰动可以判断：本模型的解对需求类参数连续、平缓地响应（线性量级、低翻转率），对预算类资源参数则在断点处发生离散跳变；稳健性结论与解析性质（命题）互为印证。

七 模型评价、改进与推广

7.1 模型的优点

- 1. **精确性**：利用最近指派引理将 p -中位模型化为 56 个集合的全枚举，获得全局最优解，彻底免除启发式算法的收敛性质疑；gap 为 0。
- 2. **可解释的结构刻画**：证明了最优值函数 $z^*(B)$ 分段常数、单调不增的性质，4 个有效断点完整刻画预算—方案—成本权衡，直接支撑决策建议。
- 3. **检验体系完整**：下界对照、贪心基线、退化检验、7 组红队对抗与三层灵敏度分析（单参数、Monte Carlo、预算扰动）相互印证，结论稳健。

7.2 模型的缺点

- 1. 距离采用欧氏近似（假设 2），未计入实际路网迂回与通行时间，绝对数值与真实配送成本存在系统性偏差。

2. 单一服务假设(假设 3)不允许片区由多站分担, 站点故障情景下的韧性不足。
3. 基础数据为假设值, 模型结论的数值部分不能直接用于实际决策, 需代入真实统计数据重新求解。

7.3 改进与推广

模型可沿三个方向推广: 引入路网迂回系数或最短路径距离替代欧氏距离; 将目标扩展为"加权距离 + 最大距离"的双目标模型(p -中位与 p -中心混合)以兼顾公平性; 在需求不确定情形下采用两阶段随机规划或鲁棒优化, 把本文的 Monte Carlo 扰动由事后检验升级为事前决策。此外, 选址—库存联合优化(location-inventory)可进一步回答"各站储备多少物资"的后续问题。

八 参考文献

- [1] Hakimi S L. Optimum locations of switching centers and the absolute centers and medians of a graph[J]. Operations Research, 1964, 12(3): 450-459.
- [2] ReVelle C S, Swain R W. Central facilities location[J]. Geographical Analysis, 1970, 2(1): 30-42.
- [3] Daskin M S. Network and Discrete Location: Models, Algorithms, and Applications[M]. 2nd ed. Hoboken: John Wiley & Sons, 2013.
- [4] 《运筹学》教材编写组. 运筹学[M]. 5 版. 北京: 清华大学出版社, 2021.
- [5] 司守奎, 孙玺菁. 数学建模算法与应用[M]. 3 版. 北京: 国防工业出版社, 2021.

九 附录

附录 A: 支撑材料文件列表

- data/: districts.csv、candidate_sites.csv (假设数据)、distance_matrix.csv (派生距离矩阵)、SOURCES.md (数据溯源)

- code/: q1_pmedian.py (问题一)、q2_budget.py (问题二与灵敏度)、sensitivity.py (扰动扫描工具)

- results/: q1_solutions.csv (56 方案全表)、q1_result.json、q1_redteam.json、q2_result.json、q2_budget_curve.csv、q2_mc.json

附录 B: 问题一核心求解代码 (q1_pmedian.py 节选)

```
1 def objective(S, w, D):
2     """z(S) = sum_i w_i * min_{j in S} d_ij (最近指派引理)"""
3     S = list(S)
4     return float((w * D[:, S].min(axis=1)).sum())
5
6 # 全枚举: 56 个组合取最小, 即全局最优
7 rows = [(S, objective(S, w, D)) for S in combinations(range(m), P)]
8 z_star = min(z for _, z in rows)
```

附录 C: 问题二预算扫描核心代码 (q2_budget.py 节选)

```
1 def solve(w, c, D, B):
2     """预算约束全枚举: 可行组合中取目标最小者"""
3     best_z, best_S = np.inf, None
4     for S in combinations(range(len(c)), P):
```

```

5         if c[list(S)].sum() <= B + TOL:
6             z = objective(S, w, D)
7             if z < best_z - TOL:
8                 best_z, best_S = z, S
9         return best_S, best_z

```

9.1 plot_setup.py

```

1  """Plot bootstrap for the managed Python runtime: sys.path + CJK fonts.
2
3  Usage in q*_.py scripts:
4      from plot_setup import plt, savefig
5      ...
6      savefig(fig, "fig1_desc.png")
7  """
8  import sys
9  from pathlib import Path
10
11 sys.path.insert(0, str(Path(sys.executable).parent.parent.parent)) # daimon_runtime
12 import matplotlib
13 matplotlib.use("Agg")
14 import matplotlib.pyplot as plt
15 from daimon_runtime import setup_plot
16
17 setup_plot()
18
19 FIGDIR = Path(__file__).resolve().parent.parent / "figures"
20 FIGDIR.mkdir(exist_ok=True)
21
22 # V3.7 图题纪律：论文用图不带图内标题—题注以文本形式写在 md 图片行
23 # 下一行（"图 N：标题"，编号全文连续），由 Word 样式统一字号。
24 # savefig 自动剥离 supitle / ax.set_title 并提醒：子图只允许 (a)/(b)
25 # 式面板小标签，用 panel_tag() 添加（豁免剥离）。
26 #
27 # V3.7.1 字号纪律：图插进 Word 会等比缩小，图内文字跟着缩水。
28 # savefig(insert_cm=...) 按"目标插入宽度"反向放大图内字号，使插入
29 # Word 缩放后图内文字 ≈10 pt（五号）；图背景默认透明（无边框、无填充色）。
30
31
32 def _strip_titles(fig, name):
33     """剥离图内标题（V3.7 纪律），panel_tag 标记的面板标签豁免。"""
34     removed = []
35     if getattr(fig, "_suptitle", None) is not None and fig._suptitle.get_text():
36         removed.append(fig._suptitle.get_text())
37         fig.suptitle("")
38     for ax in fig.axes:

```

```

39         if getattr(ax, "_v37_panel_tag", False):
40             continue
41         t = ax.get_title()
42         if t:
43             removed.append(t)
44             ax.set_title("")
45     if removed:
46         print(f"[V3.7] {name}: 图内标题已移除 {removed} — "
47               f"请在 md 中补 '图 N: <标题>' 题注行")
48     return removed
49
50
51 def _boost_fonts(fig, insert_cm, target_pt=10.0):
52     """按插入宽度反向放大图内字号: fig 越宽、插入越窄, 放大越多。
53
54     目标: PNG 插入 Word 缩到 insert_cm 宽后, 基准文字 ≈ target_pt。
55     缩放比 scale = insert_cm / fig_width_cm; 所有文字乘 1/scale,
56     再乘 target_pt/10 (rcParams 基准 10 pt)。
57
58     V3.9 防重叠:
59     - boost 封顶 1.5 — 本函数只放大字形不改布局, 过度放大必然重叠;
60     - boost>1.15 且 x 刻度 ≥6 个时旋转刻度 40°, 避免相邻刻度相撞;
61     - flow() 等自缩放图 (fig._v37_no_boost) 跳过, 禁止二次放大。
62     """
63     if getattr(fig, "_v37_no_boost", False):
64         return
65     fig_w_cm = fig.get_size_inches()[0] * 2.54
66     scale = insert_cm / max(fig_w_cm, 1e-6)
67     boost = (1.0 / scale) * (target_pt / 10.0) if scale < 0.999 else (target_pt / 10.0)
68     boost = min(boost, 1.5)
69     if abs(boost - 1.0) < 0.02:
70         return
71     for ax in fig.axes:
72         items = ([ax.title, ax.xaxis.label, ax.yaxis.label]
73                 + list(ax.get_xticklabels()) + list(ax.get_yticklabels())
74                 + list(ax.texts))
75         leg = ax.get_legend()
76         if leg is not None:
77             items += list(leg.get_texts())
78         for t in items:
79             t.set_fontsize(t.get_fontsize() * boost)
80         xt = ax.get_xticklabels()
81         if boost > 1.15 and len(xt) >= 6 and not getattr(ax, "_v37_no_rotate", False):
82             for t in xt:

```

```

83         t.set_rotation(40)
84         t.set_ha("right")
85
86
87 def _content_boxes(ax, renderer):
88     """收集轴内数据内容的 display 坐标包围盒（线/集合/补丁/文本），
89     供图例防重叠避让使用。"""
90     boxes = []
91     for ln in ax.lines:
92         xy = ln.get_xydata()
93         if len(xy) == 0:
94             continue
95         disp = ax.transData.transform(xy)
96         boxes.append((disp[:, 0].min(), disp[:, 1].min(),
97                     disp[:, 0].max(), disp[:, 1].max()))
98     for coll in ax.collections:
99         try:
100             for p in coll.get_paths():
101                 bb = p.get_extents(coll.get_transform())
102                 boxes.append((bb.x0, bb.y0, bb.x1, bb.y1))
103             except Exception:
104                 pass
105     for p in ax.patches:
106         bb = p.get_window_extent(renderer)
107         boxes.append((bb.x0, bb.y0, bb.x1, bb.y1))
108     for t in ax.texts:
109         if not t.get_text():
110             continue
111         bb = t.get_window_extent(renderer)
112         boxes.append((bb.x0, bb.y0, bb.x1, bb.y1))
113     return boxes
114
115
116 def _legend_safe(fig, name):
117     """V4.1 图例防重叠：图例与轴内内容（曲线/柱/散点/标注）相交时，
118     按候选位置重定位；图内无空位则移到轴外右侧。
119     多面板图各子图图例文本完全一致时，合并为顶部共享图例（论文惯例，
120     避免满幅面板被迫移轴外后压到相邻子图）。"""
121     try:
122         fig.canvas.draw()
123         renderer = fig.canvas.get_renderer()
124     except Exception:
125         return
126     legs = [(ax, ax.get_legend()) for ax in fig.axes

```

```

127         if ax.get_legend() is not None and ax.get_legend().get_visible()]
128     if len(legs) > 1:
129         keys = [tuple(t.get_text() for t in leg.get_texts()) for _, leg in legs]
130         if len(set(keys)) == 1 and keys[0]:
131             handles, labels = legs[0][0].get_legend_handles_labels()
132             for _, leg in legs:
133                 leg.remove()
134             fig.legend(handles, labels, loc="upper center",
135                       ncol=min(len(labels), 6), frameon=False,
136                       bbox_to_anchor=(0.5, 1.04))
137             print(f"[V4.1] {name}: 多面板重复图例 — 已合并为顶部共享图例")
138             return
139     for ax in fig.axes:
140         leg = ax.get_legend()
141         if leg is None or not leg.get_visible():
142             continue
143         boxes = _content_boxes(ax, renderer)
144         if not boxes:
145             continue
146
147         def hit(lbb):
148             for b in boxes:
149                 if not (lbb.x1 < b[0] or lbb.x0 > b[2]
150                       or lbb.y1 < b[1] or lbb.y0 > b[3]):
151                     return True
152             return False
153
154         if not hit(leg.get_window_extent(renderer)):
155             continue
156         for loc in ["upper left", "lower left", "lower right", "upper right",
157                   "center left", "center right", "lower center",
158                   "upper center", "center"]:
159             leg.set_loc(loc)
160             fig.canvas.draw()
161             if not hit(leg.get_window_extent(renderer)):
162                 print(f"[V4.1] {name}: 图例与内容重叠 — 已自动重定位到 {loc}")
163                 break
164         else:
165             leg.set_bbox_to_anchor((1.02, 1.0))
166             leg.set_loc("upper left")
167             print(f"[V4.1] {name}: 图内无空位 — 图例已移到轴外右侧")
168
169
170 _SUBSCRIPT_RE = None

```

```

171
172
173 def _lint_texts(fig, name):
174     """V4.1 伪下标 lint: 图内文字（非 mathtext）出现  $x_i / n_{\text{无风化}}$  一类
175     下划线伪下标时 WARN——要么用 mathtext  $\$n_{\{\dots\}}\$$ ，要么改写纯文字。"""
176     global _SUBSCRIPT_RE
177     if _SUBSCRIPT_RE is None:
178         import re
179         _SUBSCRIPT_RE = re.compile(r"[A-Za-z]{1,4}_[A-Za-z0-9-]{1,4}")
180     bad = set()
181     for ax in fig.axes:
182         ss = [ax.get_xlabel(), ax.get_ylabel(), ax.get_title()]
183         ss += [t.get_text() for t in ax.texts]
184         ss += [t.get_text() for t in ax.get_xticklabels()]
185         ss += [t.get_text() for t in ax.get_yticklabels()]
186         leg = ax.get_legend()
187         if leg is not None:
188             ss += [t.get_text() for t in leg.get_texts()]
189         for s in ss:
190             if not s or "$" in s:
191                 continue
192             for m in _SUBSCRIPT_RE.finditer(s):
193                 bad.add(m.group(0))
194     if bad:
195         print(f"[V4.1] {name}: 图内疑似伪下标 {sorted(bad)} — "
196               f"下标用 mathtext ( $\$n_{\{\dots\}}\$$ ) 或改写纯文字（如 未风化  $n=12$ ）")
197
198
199 def _lint_colorbar_overlap(fig, name):
200     """V4.3 colorbar 防重叠: colorbar 是独立 axes，不在 _legend_safe 的
201     Legend 检测范围内，需单独检查。重叠的经典根因: tight_layout() 写在
202     fig.colorbar() 之后（顺序应相反），或未给 pad/fraction。"""
203     try:
204         fig.canvas.draw()
205         renderer = fig.canvas.get_renderer()
206     except Exception:
207         return
208     cbs = [ax for ax in fig.axes if ax.get_label() == "<colorbar>"]
209     mains = [ax for ax in fig.axes if ax.get_label() != "<colorbar>"]
210     for cb in cbs:
211         cbb = cb.get_window_extent(renderer)
212         for ax in mains:
213             bb = ax.get_window_extent(renderer)
214             ix = max(0.0, min(cbb.x1, bb.x1) - max(cbb.x0, bb.x0))

```



```

215         iy = max(0.0, min(cbb.y1, bb.y1) - max(cbb.y0, bb.y0))
216         if ix * iy > 0.05 * cbb.width * cbb.height:
217             print(f"[V4.3] {name}: colorbar 与子图重叠 — 先 tight_layout() "
218                   f"再 fig.colorbar(..., pad=0.02), 或用 make_axes_locatable 外置")
219         return
220
221
222 def savefig(fig, name, dpi=300, insert_cm=12.0, transparent=True):
223     """保存论文用图。
224
225     insert_cm: 该图在 md 中指定的插入宽度 ({w=14cm} → insert_cm=14),
226               用于字号反缩放; 未指定时按默认档 12 cm 估算。
227     transparent: 图背景透明 (无边框、无填充色, V3.7.1 默认)。
228     """
229     _strip_titles(fig, name)
230     _boost_fonts(fig, insert_cm)
231     _legend_safe(fig, name)
232     _lint_texts(fig, name)
233     _lint_colorbar_overlap(fig, name)
234     if transparent:
235         fig.patch.set_alpha(0.0)
236         for ax in fig.axes:
237             ax.patch.set_alpha(0.0)
238     out = FIGDIR / name
239     fig.savefig(out, bbox_inches="tight", dpi=dpi, transparent=transparent)
240     print("fig saved:", out)
241     return out
242
243
244 def panel_tag(ax, tag, loc="upper left", fontsize=10):
245     """子图面板小标签 '(a) '/' (b)' — 图内唯一允许的文字标识。"""
246     ax.set_title(tag, fontsize=fontsize, loc=loc, fontweight="bold")
247     ax._v37_panel_tag = True # 标记为面板标签, _strip_titles 豁免
248     return ax
249
250
251 def paper_style(ax=None, grid=True):
252     """White-background professional style for paper figures.
253
254     Call AFTER plotting: paper_style() uses current axes when ax is None.
255     Removes top/right spines, light-gray thin grid below artists.
256     """
257     import matplotlib.pyplot as _plt
258     ax = ax or _plt.gca()

```

```

259     ax.set_facecolor("white")
260     for side in ("top", "right"):
261         ax.spines[side].set_visible(False)
262     for side in ("left", "bottom"):
263         ax.spines[side].set_color("#888888")
264         ax.spines[side].set_linewidth(0.8)
265     if grid:
266         ax.grid(True, color="#DDDDDD", lw=0.6, alpha=0.8)
267         ax.set_axisbelow(True)
268     ax.tick_params(colors="#333333", labelsz=10) # 基准 10pt (savefig 反缩放后即终稿
        字号)
269     return ax
270
271
272 def draw_circle(ax, center, r, label=None, **kw):
273     """Draw a circle (thin dark line by default). Returns the patch."""
274     import matplotlib.patches as mp
275     c = mp.Circle(center, r, fill=False, lw=1.0, color="#333333", **kw)
276     ax.add_patch(c)
277     if label:
278         ax.text(center[0], center[1], label, ha="center", va="center", fontsize=9)
279     return c
280
281
282 def mark_point(ax, p, label, color="#C00000", offset=(0.8, 0.6), s=28,
283               fontsize=10, **kw):
284     """Mark a geometry point with a label (offset in points)."""
285     ax.scatter([p[0]], [p[1]], s=s, color=color, zorder=5, **kw)
286     ax.annotate(label, (p[0], p[1]), textcoords="offset points",
287               xytext=offset, fontsize=fontsize, color=color)
288
289
290 def mark_angle(ax, vertex, p1, p2, label=None, radius=None, color="#2F5597",
291               fontsize=9, arc_kw=None):
292     """Draw an angle arc at `vertex` between rays to p1 and p2, + label."""
293     import numpy as np
294     import matplotlib.patches as mp
295     v = np.asarray(vertex, float)
296     a = np.asarray(p1, float) - v
297     b = np.asarray(p2, float) - v
298     r = radius or 0.18 * min(np.linalg.norm(a), np.linalg.norm(b))
299     t1 = np.degrees(np.arctan2(a[1], a[0]))
300     t2 = np.degrees(np.arctan2(b[1], b[0]))
301     sweep = (t2 - t1) % 360

```

```

302     if sweep > 180:
303         t1, sweep = t2, (t1 - t2) % 360
304     ax.add_patch(mp.Arc(v, 2 * r, 2 * r, angle=0, theta1=t1,
305                        theta2=t1 + sweep, color=color, lw=1.0, **(arc_kw or {})))
306     if label:
307         mid = np.radians(t1 + sweep / 2)
308         ax.text(v[0] + 1.3 * r * np.cos(mid), v[1] + 1.3 * r * np.sin(mid),
309                label, fontsize=fontsize, color=color, ha="center", va="center")
310
311
312 def flow(layers, edges, name, title=None, vgap=2.0, hgap=0.9,
313         orientation="tb", phases=None, insert_cm=12.0):
314     """Flowchart for 技术路线图 / 模型框架图 / 数据链路图.
315
316     layers: [(key, label), ...], ... # each inner list is one stage
317     edges: [(src_key, dst_key), ...] or [(src, dst, edge_label), ...]
318     name: output filename under figures/
319     orientation: "tb" stages top-down (default); "lr" stages left-to-right
320     phases: optional per-stage swimlane labels (list aligned with layers;
321            None entries skipped). Shown left of rows (tb) / above columns (lr).
322
323     Example:
324         flow([["a", "读取数据"]], [("b", "清洗"), ("c", "EDA")], [("d", "建模")],
325              [("a", "b"), ("a", "c"), ("b", "d"), ("c", "d")],
326              "fig_flow.png",
327              phases=["数据", "预处理", "建模"])
328     # V3.7 起 title 参数废弃: 图题不写进图内, 在 md 图片行下一行
329     # 写 "图 N: 技术路线图" 题注。
330     """
331     import matplotlib.patches as mpatches
332
333     tb = orientation != "lr"
334     box_h, ec, fc = 0.95, "#2F5597", "#EAF2FB"
335     label_of = {k: lab for layer in layers for k, lab in layer}
336     nst = len(layers)
337
338     def box_w(lab, k=1.0):
339         # V3.9: 盒宽随字号系数 k 同步放大, 保证放大后的文字始终装得下
340         return max(2.4 * k, 0.34 * k * len(str(lab)) + 1.0)
341
342     def layout(k=1.0):
343         """按字号系数 k 计算节点坐标与图幅 (坐标/盒宽/间距全部随 k 缩放)。"""
344         if tb:
345             main_step = vgap * k

```

```

346
347     def span(layer):
348         return sum(box_w(l, k) for _, l in layer) + hgap * k * max(len(layer) -
1, 0)
349     else:
350         main_step = max(box_w(l, k) for layer in layers for _, l in layer) +
max(hgap, 1.2) * k
351
352     def span(layer):
353         return len(layer) * (box_h + 0.55) * k - 0.55 * k
354     pos, widths, maxw = {}, {}, 0.0
355     for li, layer in enumerate(layers):
356         total = span(layer)
357         maxw = max(maxw, total)
358         u = -total / 2
359         for key, lab in layer:
360             w = box_w(lab, k)
361             if tb:
362                 pos[key] = (u + w / 2, -li * main_step)
363                 u += w + hgap * k
364             else:
365                 pos[key] = (li * main_step, -(u + box_h * k / 2))
366                 u += (box_h + 0.55) * k
367             widths[key] = w
368     if tb:
369         figw = min(max(7.5, maxw * 0.95 + (2.4 * k if phases else 0)), 16 * k)
370         figh = max(2.2, nst * main_step * 0.62 + 0.3)
371     else:
372         figw = min(max(7.5, nst * main_step * 1.0), 16 * k)
373         figh = max(2.2, maxw * 0.9 + 0.3 + (0.9 * k if phases else 0))
374     return pos, widths, maxw, main_step, figw, figh
375
376     # V3.9 自缩放: 先按基准 10pt 布局估算图宽, 推出放大系数 factor
377     # (与 _boost_fonts 同公式、同 1.5 封顶), 再按放大字号重新布局—
378     # 布局一开始就为终稿字号设计, 杜绝"先布局后放大字形"导致的重叠。
379     _, _, _, _, figw0, _ = layout(1.0)
380     factor = min(max(figw0 * 2.54 / max(insert_cm, 1e-6), 1.0), 1.5)
381     pos, widths, maxw, main_step, figw, figh = layout(factor)
382     fs = 10.0 * factor
383     k = factor
384
385     if title:
386         # V3.7: title 参数已废弃 (图题改走 md 题注行), 不再渲染也不占版面
387         print(f"[V3.7] flow(): title={title!r} 已忽略 — 请在 md 中补 '图 N: {title}' 题

```

注行")

```

388         title = None
389     fig, ax = plt.subplots(figsize=(figw, figh))
390     fig._v37_no_boost = True # 已按 factor 自缩放, _boost_fonts 不得二次放大
391     for key, (cx, cy) in pos.items():
392         w = widths[key]
393         ax.add_patch(mpatches.FancyBboxPatch(
394             (cx - w / 2, cy - box_h * k / 2), w, box_h * k,
395             boxstyle=f"round,pad={0.08 * k}", fc=fc, ec=ec, lw=1.2))
396         ax.text(cx, cy, label_of[key], ha="center", va="center", fontsize=fs)
397     for e in edges:
398         src, dst = e[0], e[1]
399         x1, y1 = pos[src]; x2, y2 = pos[dst]
400         # V4.1: 箭头起止点裁剪到盒边界, 替代固定 shrinkA/B (points 收缩
401         # 与数据坐标盒尺寸不匹配, 斜向边必戳进方框)。沿边方向求与盒
402         # 矩形 (含 round-pad) 的交点, 两端各留 0.10k 视觉间隙。
403         dx, dy = x2 - x1, y2 - y1
404         dist = max((dx * dx + dy * dy) ** 0.5, 1e-9)
405         ux, uy = dx / dist, dy / dist
406
407     def _edge_pt(key, sx, sy, vx, vy):
408         hw = widths[key] / 2 + 0.18 * k # 半宽 + pad + 间隙
409         hh = box_h * k / 2 + 0.18 * k
410         txx = hw / abs(vx) if abs(vx) > 1e-9 else float("inf")
411         tty = hh / abs(vy) if abs(vy) > 1e-9 else float("inf")
412         t = min(txx, tty)
413         return sx + vx * t, sy + vy * t
414
415     xa, ya = _edge_pt(src, x1, y1, ux, uy)
416     xb, yb = _edge_pt(dst, x2, y2, -ux, -uy)
417     ax.add_patch(mpatches.FancyArrowPatch(
418         (xa, ya), (xb, yb), arrowstyle="-|>", mutation_scale=14 * k,
419         color=ec, lw=1.1, shrinkA=0, shrinkB=0))
420     if len(e) == 3:
421         ax.text((x1 + x2) / 2 + 0.15 * k, (y1 + y2) / 2, str(e[2]),
422             fontsize=8 * k, color=ec)
423     if phases:
424         for li, ph in enumerate(phases):
425             if ph is None:
426                 continue
427             if tb:
428                 ax.text(-maxw / 2 - 1.2 * k, -li * main_step, str(ph), ha="right",
429                     va="center", fontsize=9 * k, fontweight="bold", color=ec)
430     else:

```

```

431         ax.text(li * main_step, maxw / 2 + 1.0 * k, str(ph), ha="center",
432                 fontsize=9 * k, fontweight="bold", color=ec)
433     if tb:
434         ax.set_xlim(-maxw / 2 - (2.8 * k if phases else 1.0 * k), maxw / 2 + 1.0 * k)
435         ax.set_ylim(-nst * main_step, main_step * 0.4)
436     else:
437         half = max(widths.values()) / 2 + 0.8 * k
438         ax.set_xlim(-half, (nst - 1) * main_step + half)
439         ax.set_ylim(-maxw / 2 - 1.0 * k, maxw / 2 + (2.0 * k if phases else 1.0 * k))
440     ax.axis("off")
441     return savefig(fig, name, insert_cm=insert_cm)

```

9.2 q1_pmedian.py

```

1  #!/usr/bin/env python3
2  """Q1: p-中位选址模型（无预算约束）——全枚举精确求解 + 贪心基线 + 下界检验 + 红队对抗。
3
4  输入: data/districts.csv, data/candidate_sites.csv
5  输出: data/distance_matrix.csv, results/q1_solutions.csv, results/q1_result.json,
6        results/q1_redteam.json, figures/fig1_route.png, figures/fig2_map.png
7  运行: cd job_80ad98ef && python code/q1_pmedian.py
8  """
9  import csv
10 import json
11 from itertools import combinations
12 from pathlib import Path
13
14 import numpy as np
15
16 from plot_setup import plt, savefig, paper_style, flow
17
18 ROOT = Path(__file__).resolve().parent.parent
19 P = 3 # 建站数
20 TOL = 1e-9
21
22
23 def load_data():
24     with open(ROOT / "data" / "districts.csv", encoding="utf-8") as f:
25         districts = list(csv.DictReader(f))
26     with open(ROOT / "data" / "candidate_sites.csv", encoding="utf-8") as f:
27         sites = list(csv.DictReader(f))
28     return districts, sites
29
30
31 def distance_matrix(districts, sites):
32     dxy = np.array([[float(d["x_km"]), float(d["y_km"])] for d in districts])

```

```

33     sxy = np.array([[float(s["x_km"]), float(s["y_km"])] for s in sites])
34     D = np.sqrt(((dxy[:, None, :] - sxy[None, :, :]) ** 2).sum(axis=2))
35     return D
36
37
38 def objective(S, w, D):
39     """z(S) = sum_i w_i * min_{j in S} d_ij (引理 1: 最近指派)"""
40     S = list(S)
41     return float((w * D[:, S].min(axis=1)).sum())
42
43
44 def greedy(w, D, p=P):
45     """贪心基线: 每步加入使目标下降最多的站点。"""
46     S = []
47     while len(S) < p:
48         cand = min((j for j in range(D.shape[1]) if j not in S),
49                     key=lambda j: objective(S + [j], w, D))
50         S.append(cand)
51     return S, objective(S, w, D)
52
53
54 def main():
55     districts, sites = load_data()
56     n, m = len(districts), len(sites)
57     w = np.array([float(d["demand_t_per_day"]) for d in districts])
58     names = [d["district_id"] for d in districts]
59     sids = [s["site_id"] for s in sites]
60     D = distance_matrix(districts, sites)
61
62     # 距离矩阵落盘 (派生数据, 可复现)
63     with open(ROOT / "data" / "distance_matrix.csv", "w", newline="", encoding="utf-
8") as f:
64         wr = csv.writer(f)
65         wr.writerow(["district"] + sids)
66         for i, nm in enumerate(names):
67             wr.writerow([nm] + [f"{D[i, j]:.4f}" for j in range(m)])
68
69     # 全枚举: 56 个组合
70     rows = []
71     for S in combinations(range(m), P):
72         z = objective(S, w, D)
73         rows.append((S, z))
74     rows.sort(key=lambda t: t[1])
75     z_star = rows[0][1]

```

```

76     opt_sets = [S for S, z in rows if abs(z - z_star) < TOL] # 并列最优 (良性检查项)
77
78     # 下界: 放松基数耦合, 每片区由全局最近点服务
79     lb = float((w * D.min(axis=1)).sum())
80     assert z_star >= lb - TOL, "最优值不得小于下界"
81
82     # 贪心基线
83     S_g, z_g = greedy(w, D)
84     gap = (z_g - z_star) / z_star * 100.0
85
86     # 不变量断言: 最优解的每片区恰指派一站; 建站数恰为 3
87     S0 = opt_sets[0]
88     assign = D[:, S0].argmin(axis=1)
89     assert len(S0) == P and assign.shape == (n,)
90     service = [sids[S0[a]] for a in assign]
91
92     # ---- 红队对抗 (推导稿六检查第 6 项) ----
93     red = {}
94     # R1: 某片区需求 w_i -> 0 (取 D3, 需求最小者), 最优集合中离 D3 最近的专属站应可被替换
95     w0 = w.copy(); w0[2] = 1e-6
96     z_r1 = min((objective(S, w0, D), S) for S in combinations(range(m), P))
97     red["R1_demand_to_zero"] = {"opt_set": [sids[j] for j in z_r1[1]], "z":
round(z_r1[0], 4)}
98     # R2: 两个候选点坐标完全相同 (S9 = S1 副本): 枚举不崩溃, 最优值不变,
99     # 且每个含 S1 的方案都能在"用 S9 替换 S1"后取到相同目标值
100     D2 = np.column_stack([D, D[:, 0]])
101     z_r2 = min(objective(S, w, D2) for S in combinations(range(m + 1), P))
102     paired = all(
103         any(abs(objective(tuple(j2 if j2 != 0 else m for j2 in S), w, D2) - objective(S,
w, D2)) < TOL
104             for S2_ in [tuple(m if j == 0 else j for j in S)])
105         for S in combinations(range(m), P) if 0 in S)
106     red["R2_duplicate_site"] = {"z_star": round(z_r2, 9), "equals_base": abs(z_r2 -
z_star) < TOL,
107                                "mirror_pairs_exist": bool(paired)}
108     # R3: 极远点 (1000 km 外) —任何最优解都不应选它
109     D3m = np.column_stack([D, np.full(n, 1000.0)])
110     z_r3 = min((objective(S, w, D3m), S) for S in combinations(range(m + 1), P))
111     red["R3_far_site"] = {"far_index_in_opt": (m in z_r3[1]), "z": round(z_r3[0], 4)}
112     # R4: 极端比例 (w 最大/最小 = 4.0, 本数据 48/12=4.0 已满足; 再放大到 40 倍)
113     w4 = w.copy(); w4[3] = w4[3] * 10.0 # D4 需求放大 10 倍
114     z_r4 = min((objective(S, w4, D), S) for S in combinations(range(m), P))
115     nearest_d4 = int(D[3].argmin())
116     red["R4_extreme_weight"] = {"opt_set": [sids[j] for j in z_r4[1]],

```



```

117                                     "contains_D4_nearest": nearest_d4 in z_r4[1], "z":
round(z_r4[0], 4)}
118
119     # ---- 结果落盘 ----
120     (ROOT / "results").mkdir(exist_ok=True)
121     with open(ROOT / "results" / "q1_solutions.csv", "w", newline="", encoding="utf-
8") as f:
122         wr = csv.writer(f)
123         wr.writerow(["rank", "sites", "objective_t_km_per_day"])
124         for k, (S, z) in enumerate(rows, 1):
125             wr.writerow([k, "+".join(sids[j] for j in S), f"{z:.4f}"])
126     result = {
127         "p": P, "n_districts": n, "n_sites": m, "n_combos": len(rows),
128         "optimal_sets": [[sids[j] for j in S] for S in opt_sets],
129         "z_star_t_km_per_day": round(z_star, 4),
130         "assignment": {names[i]: {"site": service[i], "dist_km": round(float(D[i,
S0[assign[i]]]), 4)} for i in range(n)},
131         "per_district_cost": {names[i]: round(float(w[i] * D[i, S0[assign[i]]]), 4)
for i in range(n)},
132         "lower_bound": round(lb, 4),
133         "greedy": {"sites": [sids[j] for j in S_g], "z": round(z_g, 4), "gap_pct":
round(gap, 4)},
134         "top5": [[("+".join(sids[j] for j in S)), round(z, 4)] for S, z in rows[:5]],
135     }
136     (ROOT / "results" / "q1_result.json").write_text(
137         json.dumps(result, ensure_ascii=False, indent=2), encoding="utf-8")
138     (ROOT / "results" / "q1_redteam.json").write_text(
139         json.dumps(red, ensure_ascii=False, indent=2), encoding="utf-8")
140
141     # ---- 图 1: 技术路线图 ----
142     flow([["a", "赛题数据与假设"],
143          [("b", "欧氏距离矩阵")],
144          [("c", "Q1: p-中位全枚举"), ("d", "Q2: 预算约束选址")],
145          [("e", "精确最优解"), ("f", "预算-目标曲线  $z^*(B)$ "),
146          [("g", "检验: 下界/贪心 gap/红队"), ("h", "灵敏度: 需求扰动与断点分析")]],
147          [("a", "b"), ("b", "c"), ("b", "d"), ("c", "e"), ("d", "f"),
148          ("e", "g"), ("f", "h"), ("g", "h")],
149          "fig1_route.png", orientation="tb")
150
151     # ---- 图 2: 选址方案地图 ----
152     fig, ax = plt.subplots(figsize=(7.2, 5.6))
153     dxy = np.array([[float(d["x_km"]), float(d["y_km"])] for d in districts])
154     sxy = np.array([[float(s["x_km"]), float(s["y_km"])] for s in sites])
155     ax.scatter(sxy[:, 0], sxy[:, 1], marker="s", s=55, color="#999999",

```

```

156         zorder=3, label="候选点（未建）")
157     ax.scatter(sxy[S0, 0], sxy[S0, 1], marker="s", s=110, color="#C00000",
158               zorder=4, label="最优建站")
159     sc = ax.scatter(dxy[:, 0], dxy[:, 1], s=w * 6, color="#2F5597", zorder=4,
160                   label="片区（大小 $\propto$ 需求）")
161     for i in range(n):
162         j = S0[assign[i]]
163         ax.plot([dxy[i, 0], sxy[j, 0]], [dxy[i, 1], sxy[j, 1]],
164               color="#C00000", lw=1.1, alpha=0.75, zorder=2)
165     for j, sid in enumerate(sids):
166         ax.annotate(sid, (sxy[j, 0], sxy[j, 1]), textcoords="offset points",
167                   xytext=(7, 7), fontsize=9)
168     for i, nm in enumerate(names):
169         ax.annotate(nm, (dxy[i, 0], dxy[i, 1]), textcoords="offset points",
170                   xytext=(7, -13), fontsize=9, color="#2F5597")
171     ax.set_xlabel("x / km"); ax.set_ylabel("y / km")
172     ax.legend(fontsize=9, loc="upper left")
173     paper_style(ax)
174     ax.set_aspect("equal")
175     savefig(fig, "fig2_map.png", insert_cm=12.0)
176
177     print(json.dumps(result, ensure_ascii=False, indent=2))
178     print("redteam:", json.dumps(red, ensure_ascii=False, indent=2))
179
180
181 if __name__ == "__main__":
182     main()

```

9.3 q2_budget.py

```

1  #!/usr/bin/env python3
2  """Q2: 预算约束 p-中位选址 — 全枚举精确求解 + 预算-目标曲线 + 灵敏度/红队。
3
4  输入: data/districts.csv, data/candidate_sites.csv, data/distance_matrix.csv
5  输出: results/q2_result.json, results/q2_budget_curve.csv, results/q2_mc.json,
6        results/q2_sens_*.md, figures/fig3_budget.png, figures/fig4_tornado.png,
figures/fig5_mc.png
7  运行: cd job_80ad98ef && python code/q2_budget.py
8  """
9  import csv
10 import json
11 from itertools import combinations
12 from pathlib import Path
13
14 import numpy as np
15

```

```

16 from plot_setup import plt, savefig, paper_style
17 from sensitivity import tornado, sweep, report
18
19 ROOT = Path(__file__).resolve().parent.parent
20 P = 3
21 B_NOMINAL = 12.0 # 名义预算(百万元)
22 TOL = 1e-9
23 SEED = 42
24
25
26 def load_all():
27     with open(ROOT / "data" / "districts.csv", encoding="utf-8") as f:
28         districts = list(csv.DictReader(f))
29     with open(ROOT / "data" / "candidate_sites.csv", encoding="utf-8") as f:
30         sites = list(csv.DictReader(f))
31     D = np.loadtxt(ROOT / "data" / "distance_matrix.csv",
32                   delimiter=",", skiprows=1, usecols=range(1, 9))
33     w = np.array([float(d["demand_t_per_day"]) for d in districts])
34     c = np.array([float(s["cost_million_yuan"]) for s in sites])
35     sids = [s["site_id"] for s in sites]
36     names = [d["district_id"] for d in districts]
37     return names, sids, w, c, D
38
39
40 def objective(S, w, D):
41     S = list(S)
42     return float((w * D[:, S].min(axis=1)).sum())
43
44
45 def solve(w, c, D, B):
46     """精确枚举。返回 (最优集合 tuple 或 None, z* 或 None)"""
47     best_z, best_S = np.inf, None
48     for S in combinations(range(len(c)), P):
49         if c[list(S)].sum() <= B + TOL:
50             z = objective(S, w, D)
51             if z < best_z - TOL:
52                 best_z, best_S = z, S
53     return best_S, (best_z if best_S is not None else None)
54
55
56 def main():
57     rng = np.random.default_rng(SEED)
58     names, sids, w, c, D = load_all()
59     m = len(sids)

```

```

60
61     # 全部组合的成本-目标表
62     combos = []
63     for S in combinations(range(m), P):
64         combos.append((S, float(c[list(S)].sum()), objective(S, w, D)))
65
66     # 名义预算求解
67     S_nom, z_nom = solve(w, c, D, B_NOMINAL)
68     assert S_nom is not None, "名义预算下不可行"
69     # 无预算约束最优 (B->inf 退化检验)
70     S_unc, z_unc = solve(w, c, D, np.inf)
71     assert z_unc <= z_nom + TOL, "加预算约束后目标不得优于无约束"
72     cost_unc = float(c[list(S_unc)].sum())
73
74     # 可行性逐条验证
75     assert len(S_nom) == P and c[list(S_nom)].sum() <= B_NOMINAL + TOL
76     assign = {names[i]: sids[S_nom[np.argmin(D[i, list(S_nom)])]] for i in
range(len(names))}
77
78     # ---- 预算-目标曲线 z*(B): Pareto 有效断点 ----
79     pts = sorted(combos, key=lambda t: t[1])
80     frontier = []
81     best = np.inf
82     for S, cost, z in pts:
83         if z < best - TOL:
84             frontier.append((cost, z, S))
85             best = z
86     b_min = min(cost for _, cost, _ in pts)
87     grid = np.round(np.arange(b_min, 15.01, 0.01), 2)
88     with open(ROOT / "results" / "q2_budget_curve.csv", "w", newline="", encoding="utf-
8") as f:
89         wr = csv.writer(f)
90         wr.writerow(["B_million", "z_star", "opt_sites"])
91         for B in grid:
92             S_b, z_b = solve(w, c, D, B)
93             wr.writerow([f"{B:.2f}", f"{z_b:.4f}", "+".join(sids[j] for j in S_b)])
94     # 单调性断言 (解析性质: 预算放松目标不减)
95     zs = [solve(w, c, D, B)[1] for B in grid]
96     assert all(zs[k] >= zs[k + 1] - TOL for k in range(len(zs) - 1)), "z*(B) 必须单调不
增"
97
98     # ---- 灵敏度 ----
99     # (a) 需求统一比例缩放 alpha: 最优集合应不变 (实现正确性校验)
100     scale_check = all(solve(w * a, c, D, B_NOMINAL)[0] == S_nom

```

```

101         for a in (0.8, 0.9, 1.1, 1.2))
102
103     # (b) Monte Carlo: 各片区需求独立 U(0.8,1.2) 倍扰动, 统计最优集合翻转
104     N_MC = 200
105     counts = {}
106     for k in range(N_MC):
107         if k % 50 == 0:
108             print(f"MC {k}/{N_MC}")
109             wp = w * rng.uniform(0.8, 1.2, size=len(w))
110             S_p, _ = solve(wp, c, D, B_NOMINAL)
111             counts["+".join(sids[j] for j in S_p)] = counts.get("+".join(sids[j] for j in
S_p), 0) + 1
112     flip_rate = 1.0 - counts.get("+".join(sids[j] for j in S_nom), 0) / N_MC
113
114     # (c) 单参数 ±20% 扫描 (sensitivity.py): 各片区需求 → z* 相对变化 → tornado
115     def mk(i):
116         def fn(r):
117             wp = w.copy(); wp[i] = r # r 为扰动后的绝对需求值
118             _, zp = solve(wp, c, D, B_NOMINAL)
119             return {"z*": zp}
120         return fn
121     multi = {"f"{nm}需求": {"res": sweep(mk(i), base=float(w[i]), label=f"{nm}需求"),
"max_dev": None}
122             for i, nm in enumerate(names)}
123     for k, v in multi.items():
124         rel = v["res"]["points"]
125         b = v["res"]["base_out"]["z*"]
126         v["max_dev"] = max(abs(p["z*"] - b) / abs(b) * 100.0 for _, p in rel)
127     tornado(multi, "fig4_tornado.png")
128
129     # (d) 预算 ±5%~20% 扫描 (报告扰动表 + 曲线)
130     def fn_B(Bv):
131         S_b, z_b = solve(w, c, D, Bv)
132         return {"z*": z_b if z_b is not None else np.nan}
133     res_B = sweep(fn_B, base=B_NOMINAL, label="预算 B / 百万元",
134                  ratios=[0.95, 0.96, 0.97, 0.98, 0.99, 1.00, 1.05, 1.10, 1.15, 1.20])
135     report(res_B, "fig6_sens_budget.png", ylabel="z* 相对变化 / %", results_dir=ROOT /
"results")
136
137     # ---- 红队 (Q2) ----
138     red = {}
139     # T1: B 恰等于断点成本 (取最小可行预算 b_min)
140     S_t1, z_t1 = solve(w, c, D, b_min)
141     red["T1_B_at_breakpoint"] = {"B": b_min, "opt": [sids[j] for j in S_t1], "z":

```

```

round(z_t1, 4)}

142     # T2: B 低于最小可行预算 → 应返回不可行而非崩溃
143     S_t2, z_t2 = solve(w, c, D, b_min - 0.5)
144     red["T2_B_below_min"] = {"feasible": S_t2 is not None}
145     # T3: 需求全 0 → 目标恒 0, 任意可行解皆最优
146     S_t3, z_t3 = solve(np.zeros_like(w), c, D, B_NOMINAL)
147     red["T3_zero_demand"] = {"z": z_t3, "opt_valid": S_t3 is not None}
148     # T4: B 恰为名义预算 ±0.0001 (断点邻域数值稳定)
149     red["T4_B_epsilon"] = {"minus": round(solve(w, c, D, B_NOMINAL - 1e-4)[1], 4),
150                             "plus": round(solve(w, c, D, B_NOMINAL + 1e-4)[1], 4)}
151
152     # ---- 结果汇总 ----
153     result = {
154         "B_nominal": B_NOMINAL,
155         "opt_sites": [sids[j] for j in S_nom],
156         "cost_million": round(float(c[list(S_nom)].sum()), 2),
157         "z_star": round(z_nom, 4),
158         "assignment": assign,
159         "unconstrained": {"sites": [sids[j] for j in S_unc], "cost": cost_unc,
160                             "z": round(z_unc, 4),
161                             "objective_loss_pct": round((z_nom - z_unc) / z_unc * 100,
162 2)},
163         "budget_min_feasible": b_min,
164         "frontier": [{"B": round(c0, 2), "z": round(z0, 4),
165                             "sites": [sids[j] for j in S0]} for c0, z0, S0 in frontier],
166         "scale_invariance_check": bool(scale_check),
167         "mc": {"n": N_MC, "seed": SEED, "flip_rate": flip_rate, "counts": counts},
168         "demand_sensitivity_max_dev_pct": {k: round(v["max_dev"], 3) for k, v in
multi.items()}},
169     }
170     (ROOT / "results" / "q2_result.json").write_text(
171         json.dumps(result, ensure_ascii=False, indent=2), encoding="utf-8")
172     (ROOT / "results" / "q2_mc.json").write_text(
173         json.dumps({"counts": counts, "flip_rate": flip_rate}, ensure_ascii=False,
indent=2),
174         encoding="utf-8")
175
176     # ---- 图 3: 预算-目标曲线 ----
177     fig, ax = plt.subplots(figsize=(7.0, 4.4))
178     ax.step(grid, zs, where="post", color="#2F5597", lw=1.6, label="$z^{*(B)}$")
179     for c0, z0, S0 in frontier:
180         ax.plot([c0], [z0], "o", ms=5, color="#C00000")
181     ax.axvline(B_NOMINAL, color="#888888", lw=1.0, ls="--")

```

```

182         ax.annotate(f"名义预算 B={B_NOMINAL}", (B_NOMINAL, zs[0]), textcoords="offset
points",
183                     xytext=(6, 0), fontsize=9, color="#555555")
184     ax.set_xlabel("预算 B / 百万元")
185     ax.set_ylabel("最优总加权距离  $z^*$  / (t·km·d $^{-1}$ )")
186     ax.legend(fontsize=9)
187     paper_style(ax)
188     savefig(fig, "fig3_budget.png", insert_cm=12.0)
189
190     # ---- 图 5: Monte Carlo 最优集合频次 ----
191     fig, ax = plt.subplots(figsize=(7.0, 3.6))
192     items = sorted(counts.items(), key=lambda kv: -kv[1])
193     labels = [k for k, _ in items]
194     vals = [v / N_MC * 100 for _, v in items]
195     colors = ["#C00000" if k == "+".join(sids[j] for j in S_nom) else "#2F5597" for k
in labels]
196     ax.bar(range(len(vals)), vals, color=colors, alpha=0.88, width=0.6)
197     ax.set_xticks(range(len(vals)))
198     ax.set_xticklabels(labels, fontsize=9)
199     ax.set_ylabel("出现频率 / %")
200     ax.set_xlabel("最优选址集合 (红 = 名义最优)")
201     paper_style(ax)
202     savefig(fig, "fig5_mc.png", insert_cm=12.0)
203
204     print(json.dumps(result, ensure_ascii=False, indent=2))
205
206
207 if __name__ == "__main__":
208     main()

```

9.4 sensitivity.py

```

1  #!/usr/bin/env python3
2  """Parametric sensitivity analysis (N4): sweep a solver over ±5%~20%
3  parameter perturbations, output a perturbation curve and a change table.
4
5  Library usage in your q*_*.py:
6      from sensitivity import sweep, report, sweep_multi, tornado
7
8      def solve(x):
9          ...
10         return {"cost": c, "time": t} # float or dict of metrics
11
12     res = sweep(solve, base=100.0, label="demand (units)")
13     report(res, "fig_sens_demand.png", ylabel="relative change / %")
14

```

```

15     # multiple parameters -> tornado chart:
16     outs = sweep_multi({"demand": solve_d, "price": solve_p}, base=1.0)
17     tornado(outs, "fig_tornado.png")
18
19 CLI self-test: python sensitivity.py --demo
20 Rules: fix random seeds inside your solver; base run is the 1.0 ratio point.
21 """
22 import argparse
23 import sys
24 from pathlib import Path
25
26 sys.path.insert(0, str(Path(__file__).resolve().parent))
27 from plot_setup import plt, savefig, paper_style # noqa: E402
28
29 RATIOS = [0.80, 0.85, 0.90, 0.95, 1.00, 1.05, 1.10, 1.15, 1.20]
30
31
32 def sweep(fn, base, ratios=None, label="parameter"):
33     """Run fn(base*r) for each ratio. fn returns float or dict[str, float]."""
34     ratios = ratios or RATIOS
35     points = []
36     for r in ratios:
37         out = fn(base * r)
38         if isinstance(out, dict):
39             points.append((r, out))
40         else:
41             points.append((r, {"value": float(out)}))
42     base_out = points[ratios.index(1.00)][1] if 1.00 in ratios else None
43     return {"label": label, "base": base, "ratios": ratios,
44           "base_out": base_out, "points": points}
45
46
47 def _rel_changes(res):
48     """{metric: [(ratio, pct_change)]} relative to the base point."""
49     metrics = res["points"][0][1].keys()
50     base = res["base_out"] or res["points"][len(res["points"]) // 2][1]
51     out = {}
52     for m in metrics:
53         b = base[m]
54         out[m] = [(r, 0.0 if b == 0 else (p[m] - b) / abs(b) * 100.0)
55                   for r, p in res["points"]]
56     return out
57
58

```



```

59 def report(res, name, ylabel="relative change / %", results_dir=None):
60     """Perturbation curve + markdown table. Returns the table text."""
61     rel = _rel_changes(res)
62     fig, ax = plt.subplots(figsize=(6.6, 4.2))
63     for m, series in rel.items():
64         ax.plot([r for r, _ in series], [v for _, v in series], "o-", ms=4, label=m)
65     ax.axhline(0, color="#888888", lw=0.8)
66     ax.set_xlabel(f"{res['label']} (ratio to base {res['base']:g})")
67     ax.set_ylabel(ylabel)
68     ax.set_title(f"Sensitivity: {res['label']}")
69     ax.legend(fontsize=9)
70     paper_style(ax)
71     savefig(fig, name)
72
73     header = "| ratio | " + " | ".join(rel.keys()) + " |"
74     sep = "|" + "---|" * (len(rel) + 1)
75     lines = [header, sep]
76     for i, r in enumerate(res["ratios"]):
77         row = f"| {r:.2f} | " + " | ".join(f"{rel[m][i][1]:+.2f}%" for m in rel) + "
| "
78         lines.append(row)
79     table = "\n".join(lines)
80     print(table)
81     if results_dir:
82         Path(results_dir).mkdir(exist_ok=True)
83         out = Path(results_dir) / (Path(name).stem + ".md")
84         out.write_text(f"# Sensitivity: {res['label']}\n\n{table}\n", encoding="utf-
8")
85     print("table saved:", out)
86     return table
87
88
89 def sweep_multi(fns, base=1.0, ratios=None):
90     """{name: fn} -> {name: max_abs_deviation_pct, series}."""
91     out = {}
92     for k, fn in fns.items():
93         res = sweep(fn, base, ratios, label=k)
94         rel = _rel_changes(res)
95         dev = max(abs(v) for series in rel.values() for _, v in series)
96         out[k] = {"res": res, "max_dev": dev}
97     return out
98
99
100 def tornado(multi, name):

```

```

101     """Tornado bar chart: max |deviation| per parameter (sorted)."""
102     items = sorted(multi.items(), key=lambda kv: kv[1]["max_dev"])
103     fig, ax = plt.subplots(figsize=(6.6, 0.7 * len(items) + 1.8))
104     ax.barh([k for k, _ in items], [v["max_dev"] for _, v in items],
105            color="#2F5597", alpha=0.85, height=0.55)
106     ax.set_xlabel("max |change| under  $\pm 20\%$  perturbation / %")
107     ax.set_title("Parameter sensitivity (tornado)")
108     paper_style(ax, grid=False)
109     ax.grid(axis="x", color="#DDDDDD", lw=0.6)
110     savefig(fig, name)
111
112
113 def _demo():
114     def fn(x):
115         return {"cost": (x - 2.0) ** 2 + 5.0, "margin": 10.0 - 0.3 * x}
116     res = sweep(fn, base=1.0, label="demo param")
117     report(res, "sens_demo.png")
118     outs = sweep_multi({"alpha": fn,
119                        "beta": lambda x: {"cost": 5 + (x - 1) ** 3, "margin": 9 - x},
120                        "gamma": lambda x: 3 + 0.1 * x})
121     tornado(outs, "sens_tornado.png")
122
123
124 if __name__ == "__main__":
125     ap = argparse.ArgumentParser()
126     ap.add_argument("--demo", action="store_true", help="run self-test")
127     args = ap.parse_args()
128     if args.demo:
129         _demo()
130     else:
131         ap.print_help()

```

9.5 stats_utils.py

```

1  """纯 numpy 统计/机器学习工具（无 scipy/sklearn 环境的自实现）
2
3  分布生存函数（Numerical Recipes 风格连分式/级数）：
4      gammq(a,x) 正则化上不完全伽马 -> chi2_sf, t_sf, f_sf
5      betai(a,b,x) 正则化不完全贝塔
6  统计检验：
7      chi2_contingency(table) -> chi2, p, dof, expected
8      cramers_v(chi2, n, r, c)
9      mannwhitney(x, y) -> U, z, p（正态近似 + 连续性修正）
10     rankdata(a)（平均秩，处理结）
11     spearman(x, y) -> rho, p（t 近似）
12     bh_fdr(pvals) -> 校后 p

```

```

13 机器学习:
14     zscore(X), clr(X, zero_repl=half-min-positive)
15     kmeans(X, k, seed, n_init) -> labels, inertia (k-means++ 初始化)
16     silhouette_score(X, labels)
17     ari(labels1, labels2)
18     cart_gini(X, y, max_depth, min_leaf) -> 规则列表 + predict 函数
19     f_ratios(X, y) -> 单变量类间/类内方差比 (特征重要性旁证)
20 贝叶斯与软聚类 (V3.6):
21     t_ppf(alpha_two_sided, df) -> t 临界值 (bisection 反解 t_sf)
22     bayes_linreg(X, y, prior) -> 后验 dict (NIG 共轭闭式解)
23     bayes_predict(post, Xnew, level) -> 均值 + 可信区间 (t 预测分布)
24     gmm(X, k, seed, n_init, cov_type) -> labels, proba, ... (EM, full/diag 协方差)
25     gmm_select(X, ks, seed) -> BIC 选簇数 {k: (bic, loglik)}
26     """
27 import math
28
29 import numpy as np
30
31 EPS = 1e-300
32
33
34 # ----- 特殊函数
35 def gammq(a, x):
36     """Regularized upper incomplete gamma Q(a,x) = P(a,x) 的补。"""
37     if x < 0 or a <= 0:
38         return float("nan")
39     if x == 0:
40         return 1.0
41     if x < a + 1.0: # 级数表示 P, 再取补
42         ap, s, d = a, 1.0 / a, 1.0 / a
43         for _ in range(1000):
44             ap += 1
45             d *= x / ap
46             s += d
47             if abs(d) < abs(s) * 1e-14:
48                 break
49         p = s * math.exp(-x + a * math.log(x) - math.lgamma(a))
50         return max(0.0, 1.0 - p)
51     # 连分式表示 Q
52     tiny = 1e-300
53     b, c, d = x + 1 - a, 1 / tiny, 1 / (x + 1 - a)
54     h = d
55     for i in range(1, 1000):
56         an = -i * (i - a)

```

```

57         b += 2
58         d = an * d + b
59         if abs(d) < tiny:
60             d = tiny
61         c = b + an / c
62         if abs(c) < tiny:
63             c = tiny
64         d = 1 / d
65         delta = d * c
66         h *= delta
67         if abs(delta - 1) < 1e-14:
68             break
69     return max(0.0, h * math.exp(-x + a * math.log(x) - math.lgamma(a)))
70
71
72 def betai(a, b, x):
73     """Regularized incomplete beta I_x(a,b) (连分式)。"""
74     if x <= 0:
75         return 0.0
76     if x >= 1:
77         return 1.0
78     lbeta = (math.lgamma(a + b) - math.lgamma(a) - math.lgamma(b)
79             + a * math.log(x) + b * math.log(1 - x))
80     front = math.exp(lbeta)
81
82     def cf(a_, b_, x_):
83         tiny = 1e-300
84         qab, qap, qam = a_ + b_, a_ + 1, a_ - 1
85         c, d = 1.0, 1.0 - qab * x_ / qap
86         if abs(d) < tiny:
87             d = tiny
88         d = 1 / d
89         h = d
90         for m in range(1, 1000):
91             m2 = 2 * m
92             aa = m * (b_ - m) * x_ / ((qam + m2) * (a_ + m2))
93             d = 1 + aa * d
94             if abs(d) < tiny:
95                 d = tiny
96             c = 1 + aa / c
97             if abs(c) < tiny:
98                 c = tiny
99             d = 1 / d
100            h *= d * c

```

```

101         aa = -(a_ + m) * (qab + m) * x_ / ((a_ + m2) * (qap + m2))
102         d = 1 + aa * d
103         if abs(d) < tiny:
104             d = tiny
105         c = 1 + aa / c
106         if abs(c) < tiny:
107             c = tiny
108         d = 1 / d
109         delta = d * c
110         h *= delta
111         if abs(delta - 1) < 1e-14:
112             break
113     return h
114
115     if x < (a + 1) / (a + b + 2):
116         return front * cf(a, b, x) / a
117     return 1 - front * cf(b, a, 1 - x) / b
118
119
120 def chi2_sf(x, df):
121     return gammq(df / 2.0, x / 2.0)
122
123
124 def t_sf(t, df):
125     """双侧 |T| > |t| 的概率。"""
126     x = df / (df + t * t)
127     return betai(df / 2.0, 0.5, x)
128
129
130 def f_sf(f, d1, d2):
131     x = (d1 * f) / (d1 * f + d2)
132     return 1.0 - betai(d1 / 2.0, d2 / 2.0, x)
133
134
135 def norm_sf(z):
136     return 0.5 * math.erfc(z / math.sqrt(2))
137
138
139 # ----- 检验
140 def chi2_contingency(table):
141     obs = np.asarray(table, dtype=float)
142     n = obs.sum()
143     rs, cs = obs.sum(1, keepdims=True), obs.sum(0, keepdims=True)
144     exp = rs @ cs / n

```

```

145     mask = exp > 0
146     chi2 = float(((obs - exp) ** 2 / np.where(mask, exp, 1))[mask].sum())
147     dof = (obs.shape[0] - 1) * (obs.shape[1] - 1)
148     return chi2, chi2_sf(chi2, dof), dof, exp
149
150
151 def cramers_v(chi2, n, r, c):
152     return math.sqrt(chi2 / (n * max(1, min(r - 1, c - 1))))
153
154
155 def mannwhitney(x, y):
156     x, y = np.asarray(x, float), np.asarray(y, float)
157     n1, n2 = len(x), len(y)
158     r = rankdata(np.concatenate([x, y]))
159     r1 = r[:n1].sum()
160     u1 = r1 - n1 * (n1 + 1) / 2
161     mu = n1 * n2 / 2
162     # 结的修正
163     _, counts = np.unique(np.concatenate([x, y]), return_counts=True)
164     tie = (counts ** 3 - counts).sum()
165     sigma = math.sqrt(n1 * n2 / 12 * (n1 + n2 + 1 - tie / ((n1 + n2) * (n1 + n2 - 1))))
166     z = (u1 - mu - 0.5 * np.sign(u1 - mu)) / sigma
167     return u1, z, 2 * norm_sf(abs(z))
168
169
170 def rankdata(a):
171     a = np.asarray(a, float)
172     order = a.argsort(kind="mergesort")
173     ranks = np.empty(len(a))
174     sa = a[order]
175     i = 0
176     while i < len(a):
177         j = i
178         while j + 1 < len(a) and sa[j + 1] == sa[i]:
179             j += 1
180         ranks[order[i:j + 1]] = (i + j) / 2 + 1
181         i = j + 1
182     return ranks
183
184
185 def spearman(x, y):
186     x, y = np.asarray(x, float), np.asarray(y, float)
187     m = ~(np.isnan(x) | np.isnan(y))
188     x, y = x[m], y[m]

```

```

189     n = len(x)
190     rx, ry = rankdata(x), rankdata(y)
191     rx, ry = rx - rx.mean(), ry - ry.mean()
192     denom = math.sqrt((rx ** 2).sum() * (ry ** 2).sum())
193     rho = float((rx * ry).sum() / denom) if denom > 0 else 0.0
194     if abs(rho) >= 1:
195         return rho, 0.0
196     t = rho * math.sqrt((n - 2) / (1 - rho ** 2))
197     return rho, t_sf(t, n - 2)
198
199
200 def bh_fdr(pvals):
201     p = np.asarray(pvals, float)
202     n = len(p)
203     order = p.argsort()
204     ranked = p[order] * n / (np.arange(n) + 1)
205     ranked = np.minimum.accumulate(ranked[::-1])[:-1]
206     out = np.empty(n)
207     out[order] = np.minimum(ranked, 1.0)
208     return out
209
210
211 # ----- 变换与 ML
212 def zscore(X):
213     X = np.asarray(X, float)
214     sd = X.std(0)
215     sd[sd == 0] = 1
216     return (X - X.mean(0)) / sd
217
218
219 def clr(X):
220     """Centered log-ratio: 0 值替换为该列最小正值的一半。"""
221     X = np.asarray(X, float).copy()
222     for k in range(X.shape[1]):
223         col = X[:, k]
224         pos = col[col > 0]
225         repl = pos.min() / 2 if len(pos) else 1e-3
226         col[col <= 0] = repl
227         X[:, k] = col
228     lg = np.log(X)
229     return lg - lg.mean(1, keepdims=True)
230
231
232 def kmeans(X, k, seed=0, n_init=10, max_iter=300):

```

```

233     X = np.asarray(X, float)
234     best = None
235     for init in range(n_init):
236         rng = np.random.default_rng(seed * 1000 + init)
237         idx = [rng.integers(len(X))]
238         for _ in range(k - 1):
239             d2 = np.min(((X[:, None, :] - X[idx][None]) ** 2).sum(-1), 1)
240             idx.append(rng.choice(len(X), p=d2 / d2.sum()))
241         C = X[idx].copy()
242         for _ in range(max_iter):
243             lab = ((X[:, None, :] - C[None]) ** 2).sum(-1).argmin(1)
244             Cn = np.array([X[lab == i].mean(0) if (lab == i).any() else C[i]
245                             for i in range(k)])
246             if np.allclose(Cn, C):
247                 break
248             C = Cn
249             inertia = float(((X - C[lab]) ** 2).sum())
250             if best is None or inertia < best[2]:
251                 best = (lab, C, inertia)
252     return best
253
254
255 def silhouette_score(X, labels):
256     X = np.asarray(X, float)
257     D = np.sqrt(((X[:, None, :] - X[None]) ** 2).sum(-1))
258     s = np.zeros(len(X))
259     for i in range(len(X)):
260         same = (labels == labels[i])
261         same[i] = False
262         a = D[i, same].mean() if same.any() else 0.0
263         b = min(D[i, labels == c].mean() for c in set(labels) if c != labels[i])
264         s[i] = (b - a) / max(a, b) if max(a, b) > 0 else 0.0
265     return float(s.mean())
266
267
268 def ari(a, b):
269     a, b = np.asarray(a), np.asarray(b)
270     n = len(a)
271
272     def comb2(x):
273         return x * (x - 1) / 2
274
275     tab = np.zeros((a.max() + 1, b.max() + 1))
276     for i in range(n):

```



```

277         tab[a[i], b[i]] += 1
278     sum_c = comb2(tab).sum()
279     sa, sb = comb2(tab.sum(1)).sum(), comb2(tab.sum(0)).sum()
280     total = comb2(n)
281     expected = sa * sb / total
282     maxi = (sa + sb) / 2
283     return (sum_c - expected) / (maxi - expected) if maxi != expected else 1.0
284
285
286 def cart_gini(X, y, max_depth=3, min_leaf=3):
287     """二分类 CART (gini), 返回规则列表与预测函数。"""
288     X, y = np.asarray(X, float), np.asarray(y)
289     rules = []
290
291     def gini(idx):
292         if len(idx) == 0:
293             return 0.0
294         p = (y[idx] == y[0]).mean()
295         # 二分类通用: 两类比例
296         classes, counts = np.unique(y[idx], return_counts=True)
297         ps = counts / len(idx)
298         return float(1 - (ps ** 2).sum())
299
300     def split(idx, depth, path):
301         maj = y[idx][0] if (y[idx] == y[0]).mean() >= 0.5 else [c for c in np.unique(y)
if c != y[0]][0]
302         counts = {c: int((y[idx] == c).sum()) for c in np.unique(y)}
303         node = {"path": list(path), "n": len(idx), "counts": counts,
304                 "label": max(counts, key=counts.get)}
305         if depth >= max_depth or len(idx) < 2 * min_leaf or len(counts) == 1:
306             rules.append(node)
307             return node["label"]
308         best = None
309         for k in range(X.shape[1]):
310             vals = np.unique(X[idx, k])
311             if len(vals) < 2:
312                 continue
313             thr_cand = (vals[:-1] + vals[1:]) / 2
314             for t in thr_cand:
315                 l, r = idx[X[idx, k] <= t], idx[X[idx, k] > t]
316                 if len(l) < min_leaf or len(r) < min_leaf:
317                     continue
318                 g = len(l) / len(idx) * gini(l) + len(r) / len(idx) * gini(r)
319                 if best is None or g < best[0]:

```

```

320         best = (g, k, t)
321     if best is None:
322         rules.append(node)
323         return node["label"]
324     _, k, t = best
325     l, r = idx[X[idx, k] <= t], idx[X[idx, k] > t]
326     left = split(l, depth + 1, path + [(k, "<=", t)])
327     right = split(r, depth + 1, path + [(k, ">", t)])
328     return node["label"]
329
330 split(np.arange(len(X)), 0, [])
331 return rules
332
333
334 def f_ratios(X, y):
335     """单变量类间/类内方差比（越大区分度越高）。"""
336     X, y = np.asarray(X, float), np.asarray(y)
337     out = []
338     for k in range(X.shape[1]):
339         grand = X[:, k].mean()
340         between = sum(((y == c).sum()) * (X[y == c, k].mean() - grand) ** 2
341                       for c in np.unique(y))
342         within = sum(((X[y == c, k] - X[y == c, k].mean()) ** 2).sum()
343                     for c in np.unique(y))
344         out.append(between / within if within > 0 else np.inf)
345     return np.array(out)
346
347
348 # ----- 贝叶斯回归
349 def t_ppf(alpha_two_sided, df):
350     """双侧 alpha 的 t 临界值：求 t 使  $P(|T| > t) = \alpha$  (bisection 反解 t_sf)。"""
351     lo, hi = 0.0, 1.0
352     while t_sf(hi, df) > alpha_two_sided:
353         hi *= 2
354         if hi > 1e6:
355             break
356     for _ in range(200):
357         mid = (lo + hi) / 2
358         if t_sf(mid, df) > alpha_two_sided:
359             lo = mid
360         else:
361             hi = mid
362     return (lo + hi) / 2
363

```

```

364
365 def bayes_linreg(X, y, prior=None):
366     """共轭贝叶斯线性回归 (Normal-Inverse-Gamma, 闭式解)。
367
368     模型  $y = X\beta + \varepsilon$ ,  $\varepsilon \sim N(0, \sigma^2)$ ; 先验  $\beta|\sigma^2 \sim N(m0, \sigma^2 V0)$ ,  $\sigma^2 \sim \text{InvGamma}(a0, b0)$ 。
369     默认弱信息先验 ( $V0=1e6 \cdot I$ ,  $a0=b0=1e-3$ )  $\rightarrow$  后验均值=OLS, 区间含贝叶斯修正。
370     prior 可传 dict( $m0=$ ,  $V0=$ ,  $a0=$ ,  $b0=$ ) 覆盖; X 需自行加截距列。
371     返回 dict( $mn$ ,  $Vn$ ,  $an$ ,  $bn$ ):  $\beta|y \sim t_{\{2an\}}(mn, bn/an \cdot Vn)$ 。
372     """
373     X, y = np.asarray(X, float), np.asarray(y, float)
374     d = X.shape[1]
375     pr = {"m0": np.zeros(d), "V0": np.eye(d) * 1e6, "a0": 1e-3, "b0": 1e-3}
376     if prior:
377         pr.update(prior)
378     m0, V0, a0, b0 = (np.asarray(pr["m0"], float), np.asarray(pr["V0"], float),
379                       pr["a0"], pr["b0"])
380     V0i = np.linalg.inv(V0)
381     Vn = np.linalg.inv(V0i + X.T @ X)
382     mn = Vn @ (V0i @ m0 + X.T @ y)
383     an = a0 + len(y) / 2
384     bn = b0 + 0.5 * (y @ y - mn @ np.linalg.inv(Vn) @ mn + m0 @ V0i @ m0)
385     return {"mn": mn, "Vn": Vn, "an": an, "bn": bn}
386
387
388 def bayes_predict(post, Xnew, level=0.95):
389     """后验预测: 返回 (mean, lo, hi)。预测分布为  $t_{\{2an\}}(x'mn, s^2)$ ,  $s^2 = bn/an \cdot (1 + x'Vn$ 
x)"""
390     Xnew = np.atleast_2d(np.asarray(Xnew, float))
391     mn, Vn, an, bn = post["mn"], post["Vn"], post["an"], post["bn"]
392     mean = Xnew @ mn
393     s2 = bn / an * (1 + np.einsum("ij,jk,ik->i", Xnew, Vn, Xnew))
394     tcrit = t_ppf(1 - level, 2 * an)
395     half = tcrit * np.sqrt(s2)
396     return mean, mean - half, mean + half
397
398
399 # ----- GMM 软聚类
400 def _gauss_logpdf(X, mean, cov, reg):
401     d = X.shape[1]
402     cov_r = cov + np.eye(d) * reg
403     sign, logdet = np.linalg.slogdet(cov_r)
404     if sign <= 0: # 协方差退化, 加大正则
405         cov_r = cov + np.eye(d) * reg * 100
406         sign, logdet = np.linalg.slogdet(cov_r)

```

```

407     sol = np.linalg.solve(cov_r, (X - mean).T).T
408     maha = ((X - mean) * sol).sum(1)
409     return -0.5 * (d * math.log(2 * math.pi) + logdet + maha)
410
411
412 def gmm(X, k, seed=0, n_init=5, max_iter=200, reg=1e-6, tol=1e-6,
413        cov_type="full"):
414     """高斯混合模型 (EM)。k-means 初始化, 多次重启取最优。
415
416     cov_type: "full" 全协方差 (需  $n \gg k \cdot d^2 / 2$ , 否则过参数化);
417             "diag" 对角协方差 (小样本纪律:  $n$  有限时的默认稳妥选择)。
418     返回 (labels, proba, weights, means, covs, loglik):
419         proba (n,k) 后验归属概率— $\max(\text{proba}) < 0.7$  的样本即"过渡样本"。
420     """
421     X = np.asarray(X, float)
422     n, d = X.shape
423     diag = cov_type == "diag"
424     best = None
425     for init in range(n_init):
426         km_lab, C0, _ = kmeans(X, k, seed=seed * 100 + init)
427         means = C0.copy()
428         covs = np.array([np.cov(X[km_lab == i].T) if (km_lab == i).sum() > 1
429                          else np.eye(d) * X.var(0).mean() for i in range(k)])
430         if diag:
431             covs = np.array([np.diag(np.maximum(np.diag(c), 1e-9)) for c in covs])
432         w = np.array([(km_lab == i).mean() for i in range(k)])
433         w = np.maximum(w, 1e-6)
434         w /= w.sum()
435         ll_old = -np.inf
436         for _ in range(max_iter):
437             # E
438             logp = np.column_stack([
439                 _gauss_logpdf(X, means[i], covs[i], reg) + math.log(w[i])
440                 for i in range(k)])
441             logp -= logp.max(1, keepdims=True)
442             proba = np.exp(logp)
443             proba /= proba.sum(1, keepdims=True)
444             # 标准对数似然 (log-sum-exp)
445             comp = np.column_stack([
446                 _gauss_logpdf(X, means[i], covs[i], reg) for i in range(k)])
447             lw = comp + np.log(np.maximum(w, EPS))
448             mx = lw.max(1, keepdims=True)
449             ll = float((mx[:, 0] + np.log(np.exp(lw - mx).sum(1))).sum())
450             # M

```

```

451         Nk = proba.sum(0)
452         empty = Nk < 1e-8
453         if empty.any(): # 空簇重置到最不适配的样本
454             worst = proba.max(1).argmin()
455             for i in np.where(empty)[0]:
456                 means[i] = X[worst]
457                 covs[i] = np.eye(d) * X.var(0).mean()
458                 if diag:
459                     covs[i] = np.diag(np.full(d, X.var(0).mean()))
460                 Nk[i] = 1.0
461                 proba[worst] = 0
462                 proba[worst, i] = 1.0
463         w = Nk / n
464         means = (proba.T @ X) / Nk[:, None]
465         for i in range(k):
466             dev = X - means[i]
467             covs[i] = (dev * proba[:, i:i + 1]).T @ dev / Nk[i]
468             if diag:
469                 covs[i] = np.diag(np.maximum(np.diag(covs[i]), 1e-9))
470             if abs(ll - ll_old) < tol * (1 + abs(ll)):
471                 break
472             ll_old = ll
473         labels = proba.argmax(1)
474         if best is None or ll > best[-1]:
475             best = (labels, proba, w, means, covs, ll)
476     return best
477
478
479 def gmm_select(X, ks, seed=0, n_init=5, cov_type="full"):
480     """BIC 选簇数: 返回 {k: (bic, loglik)}, BIC 越小越好。"""
481     X = np.asarray(X, float)
482     n, d = X.shape
483     out = {}
484     for k in ks:
485         _, _, _, _, ll = gmm(X, k, seed=seed, n_init=n_init,
486                               cov_type=cov_type)
487         pcov = d if cov_type == "diag" else d * (d + 1) / 2
488         p = k * (d + pcov) + (k - 1) # 均值+协方差+权重
489         out[k] = (-2 * ll + p * math.log(n), ll)
490     return out

```